PROJECT IMPLEMENTATION

IMPLEMENT SPRING SECURITY

1. **Add Dependencies** - spring-boot-starter-security, spring-boot-starter-web, jsonwebtoken (api, impl, Jackson)
2. Add a **User Model that implements UserDetails**. **Spring Security needs** to **know which** field is **the "username" and which is the "password**" to perform the internal checks.
3. **Add JwtService that handles logic for creating and reading the tokens**. You will need a **Signing Key** (a long random string).
4. **Create a JWT Filter class that intercepts the incoming request and checks Bearer token in the header.** If there is, it validates it and tells Spring Security.
5. **Add a Security Configuration Class Annotated with @Configuration and @EnableWebSecurity.**This tell Spring to disable sessions (making it stateless) and to use your custom filter.
6. **Inside Security Configuration class Implement SecurityFilterChain menthod**, (this accepts a HttpSecurity object, **that allows you to fine what urls or endpoint a particular use can access**. Inside the securityFilterChain you also add the JwtFilter class inside the addFilterBefore.
7. Use it in **the login endpoint**. If the user logins it will call **the JwtService to generatetoken.**

"Why do you do this?", mention that it moves the app from **Stateful** (server-side sessions) to **Stateless** (client-side tokens), which allows your API to scale horizontally across multiple servers.

SECURITY In MICROSERVICES

Gateway

```
public class AuthorizationHeaderFilter extends
AbstractGatewayFilterFactory<AuthorizationHeaderFilter.Config> {

   @Autowired
   JWTUtil jwtUtil;

   @Override
   public GatewayFilter apply(Config config) {}
}
```

Member Service

```
@EnableWebSecurity
public class SecurityConfig {
 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {}

// 1. Modern Data Carrier (Java 21 Record)
public record User(Long id, String name) {}

// 2. The Service we are testing
@Service
public class UserService {
   private final UserRepository repository;

   public UserService(UserRepository repository) {
```

```
    this.repository = repository;
  }

  public User createUser(String name) {
    if (name == null || name.isBlank()) throw new IllegalArgumentException("Name required");
    return repository.save(new User(null, name));
  }
}
```

UNIT TEST

Layer	Java (Spring) Tool	Angular Tool
Framework	JUnit 5	Jasmine
Runner	Maven/Gradle	Karma (or Vitest/Jest)
Mocking	Mockito	Spies
Logic Check	assertEquals()	expect().toBe()

- **Red-Green-Refactor**: Mention this as the TDD cycle.
- **Mocks vs Spies**: In Java we "mock" objects; in Angular we "spy" on them.
- **Isolation**: Unit tests should never call real APIs or Databases.

Interview A good Unit Test -

What is a good unit test:

Script:

- Follows **AAA** pattern. It should be broken into three distinct blocks:
  - **Arrange:** Set up the objects and the data you need.
  - **Act:** Call the specific method you are testing.
  - **Assert:** Verify that the result is what you expected.
- Is readable and uses descriptive naming so that when a failure occurs, the name alone tells me exactly what broke and why (like **shouldReturnErrorWhenPasswordIsTooShort** instead of **test1)**
- Unit tests should never call real APIs or Databases (Use Mocks for it)

What is a good unit test coverage:

I aim for an industry-standard 80% coverage because chasing 100% often leads to diminishing returns, where you waste time testing trivial boilerplate instead of critical logic. I prioritize meaningful coverage over high numbers, ensuring that complex business rules and edge cases are thoroughly verified rather than just following the 'happy path'.

What is a unit test.

- A unit test verifies the smallest isolated piece of logic—like a **Java method** or an **Angular component**—using **JUnit** or **Jasmine** to ensure the code works correctly without relying on external databases or APIs. By using mocks to replace those dependencies, I can guarantee that my backend business rules and frontend UI logic are robust and reliable in isolation.

Java Unit Test

- A unit test checks a single "unit" of code (usually one method) in isolation.
- In testing service, your service depends on database repository. You don't want to call the database in unit test, so You mock it.

Interview:

- **Mention the Pyramid**: If they ask about your testing strategy, mention the Testing Pyramid. You want many Unit Tests, fewer Integration Tests, and even fewer UI (End-to-End) tests because unit tests are cheaper and faster.
- **TDD (Test Driven Development):** If you want to impress them, mention TDD. It's the practice of writing the test before the actual code (Red-Green-Refactor).
- **Angular Side:** Since you mentioned Angular, they might ask about Jasmine and Karma. Mention that the logic is the same (Arrange/Act/Assert), just in TypeScript.

Code:

```
@ExtendWith(MockitoExtension.class) // Initializes mocks
class UserServiceTest {

  @Mock
  private UserRepository userRepository; // The fake dependency

  @InjectMocks
  private UserService userService; // The class we are actually testing

  @Test
  void testGetUserById() {
    // Arrange: Tell the mock what to return when called
    User mockUser = new User(1, "John Doe");
    when(userRepository.findById(1)).thenReturn(java.util.Optional.of(mockUser));

    // Act
    User result = userService.getUser(1);

    // Assert
    assertNotNull(result);
    assertEquals("John Doe", result.getName());

    // Verify: Ensure the repository was actually called once
    verify(userRepository, times(1)).findById(1);
  }
}
```

Angular Unit Test

counter.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <h1 id="count-display">Count: {{ count }}</h1>
    <button (click)="increment()" id="increment-btn">Add</button>
  `
})
export class CounterComponent {
  count = 0;
```

```
  increment() {
    this.count++;
  }
}
```

counter.component.spec.ts

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { CounterComponent } from './counter.component';

describe('CounterComponent', () => {
  let component: CounterComponent;
  let fixture: ComponentFixture<CounterComponent>;

  beforeEach(async () => {
    // 1. Setup the testing module
    await TestBed.configureTestingModule({
      imports: [CounterComponent]
    }).compileComponents();

    fixture = TestBed.createComponent(CounterComponent);
    component = fixture.componentInstance; // The TS Class
    fixture.detectChanges(); // Syncs the UI
  });

  // TEST 1: Logic Test
  it('should increase the count variable when increment() is called', () => {
    component.increment();
    expect(component.count).toBe(1);
  });

  // TEST 2: UI Test (DOM)
  it('should update the H1 tag in the HTML when the button is clicked', () => {
    // Find the button and click it
    const button = fixture.nativeElement.querySelector('#increment-btn');
    button.click();

    // Tell Angular to update the HTML display
    fixture.detectChanges();

    // Check the H1 text
    const h1 = fixture.nativeElement.querySelector('#count-display');
    expect(h1.textContent).toContain('Count: 1');
  });
});
```

In a microservices world, if you have 50 services and you need to change a database URL, doing it manually for each one is a nightmare. This is why we use a **Configuration Server** (like Spring Cloud Config).

Spring Cloud Config

Centralized Management - all configurations for all services (and all environments) are kept in one central place.

Dynamic Refresh (No Restarts) - One of the "superpowers" of Spring Cloud Config is the ability to update properties at runtime.

ExCEPTION HANDLING SPRING

The most common approach is using a combination of @RestControllerAdvice and @ExceptionHandler

 Define a Custom Error Response: Create a simple POJO or record to define what the client sees.
 Create the Global Handler: A class annotated with @RestControllerAdvice.

```
// 1. Consistent Error Structure
public record ErrorResponse(int status, String message, long timestamp) {}

// 2. Global Exception Handler
@RestControllerAdvice
public class GlobalExceptionHandler {

  // Handles specific business exceptions
  @ExceptionHandler(UserNotFoundException.class)
  public ResponseEntity<ErrorResponse> handleUserNotFound(UserNotFoundException ex) {
    ErrorResponse error = new ErrorResponse(
      HttpStatus.NOT_FOUND.value(),
      ex.getMessage(),
      System.currentTimeMillis()
    );
    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
  }

  // Handles generic validation/bad request exceptions
  @ExceptionHandler(IllegalArgumentException.class)
  public ResponseEntity<ErrorResponse> handleBadRequest(IllegalArgumentException ex) {
    return ResponseEntity.badRequest().body(
      new ErrorResponse(400, ex.getMessage(), System.currentTimeMillis())
    );
  }
}
```

-

Don't forget that **Bean Validation** (e.g., @Valid, @NotNull) throws a MethodArgumentNotValidException. In a real interview, mention that you would override handleMethodArgumentNotValid in your Global Handler to extract and return the specific field errors (e.g., "Email is invalid").